Team Control Number

16843

Problem Chosen

A

2025
HiMCM/MidMCM
Summary Sheet

Race the Blaze, Carve the Ways

Emergencies unfold with little warning, giving rise to dire situations where every second is life-changing. Safe and timely evacuation requires responders to navigate complex factors, including structure layout, nature of residence, and responder capabilities. This paper proposes an effective mathematical model "**RACE-S**", the Responder Action and Clearance Evacuation Solver, to determine the optimal sweeping strategy under various scenarios, aiming to save more time, effort, and lives.

We model the building as a **time-dependent probabilistic graph simulator**. Nodes for rooms describe probabilistic distributions of room service time, including entry, inspection, and assistance time, accounting for realistic factors such as locked doors, clutter, calming and evacuating occupants, and in particular **uncertainty and time-evolution of ambulatory/non-ambulatory occupant counts**. Each node is also assigned a risk weight prioritizing rooms with possible vulnerable occupants and high hazard proximity. Edges encode travel time between nodes that increases when smoke spreads or when responders carry non-ambulatory occupants.

On this representation we build a real-time sweep path planner that **combines greedy route construction with local neighborhood search strategy**, repeatedly replanning as new room information is revealed, dynamically minimizing the statistical expectation of sweep time while prioritizing high-risk rooms. Furthermore, to determine the optimal number and location of responders under uncertainty, we implement **a greedy algorithm for responder location decision**, which maximizes coverage of rooms considering prioritization, and a **Multi-Armed Bandit framework to efficiently select the optimal number of responders under computational budget**, using the Upper Confidence Bound policy, minimizing sweep time with fewer responder hired.

We then apply RACE-S to three layouts that are **a simple office**, a complex **hospital inpatient ward** and a **multi-use office building**. For each, we estimate sweep time, optimal responder numbers, and starting locations, and show diminishing returns beyond these recommended team sizes. For the hospital layout, where patient rooms are the majority and most occupants need assistance, four responders **finish the sweep in less than 5 minutes**; for the more complex three-floor office building, where room types vary greatly and occupants are harder to predict, **six responders end the sweep in only 6.5 minutes**. Responders remain engaged and complete their tasks within a tight interval, visiting high-probability rooms as soon as possible. The sweep strategy model also yields outstanding results, which we showcase clearly in visualization maps.

Finally, we conduct a sensitivity analysis of our dynamic sweep path and responder setting planner by repeatedly sampling ground truth building states. For sweeping paths, despite substantial variation in room conditions, **the overall sweep-time overhead remains below** 25%; for responder setting optimization, 128 random runs prove to be sufficient for estimates and recommendations, while keeping computational cost manageable. We then extend the model to **odorless gas leaks** to showcase how our model accounts for realistic factors, and discuss how technologies such as sensors, building information models, and AR guidance can further reduce sweep time and enhance safety levels.

Taken together, our results demonstrate that RACE-S produces adaptable, quantitatively efficient sweep strategies that prioritize high-risk rooms, support practical planning and are ready for action.

Keywords: Emergency planning, Graph representation, Greedy algorithm, Local search, Multiarmed bandit.

Contents

1 Introduction **3**
- 1.1 Background . . . 3
- 1.2 Problem Restatement . . . 3
- 1.3 Our Work . . . 3
- 1.4 Summary of Factors . . . 4

2 Assumptions and Justifications **4**

3 Simulating Buildings and Occupants Using Graphs **5**
- 3.1 Describing a Building with a Graph Structure . . . 5
- 3.2 Room Nodes R Description . . . 6
  - 3.2.1 Number of Occupants in a Room . . . 6
  - 3.2.2 Service Time T_s^i Spent on a Room i . . . 9
  - 3.2.3 Risk Weight of a Room . . . 10
- 3.3 Edges E Description: Transit Time T_w^e Spent on an Edge e . . . 11

4 RACE-S: Real-time Planner for Sweeping Path and Responder Setup **12**
- 4.1 Optimization for Sweeping Paths . . . 12
  - 4.1.1 Objective . . . 12
  - 4.1.2 Solver Based on Greedy Initialization and Local Neighborhood Search . . . 12
- 4.2 Optimization for Responder Setting . . . 14
  - 4.2.1 Objective . . . 14
  - 4.2.2 Solver Based on Greedy Room Selection with Adaptive Simulation . . . 14

5 Results of Responders Reacting to a Fire Alarm **17**
- 5.1 Layout 1: The Provided Floor Plan of an Office . . . 17
- 5.2 Layout 2: A Hospital Building . . . 18
- 5.3 Layout 3: Multi-use Office Building . . . 19
- 5.4 Sensitivity Analysis . . . 20
  - 5.4.1 Sensitivity of Sweep Path Optimization . . . 20
  - 5.4.2 Sensitivity of Responder Setting Optimization . . . 20

6 Realistic Factors and Additional Technologies **21**
- 6.1 Parametric study: Odorless Gas Leakage . . . 21
- 6.2 Additional Technologies . . . 22

7 Advantages and Weaknesses **22**

8 Conclusion **23**

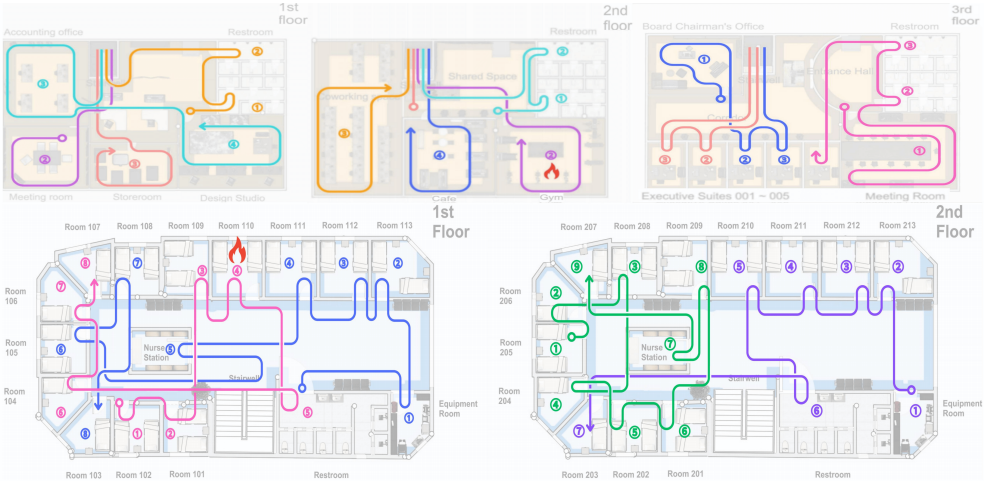
Dear Local Emergency Planning Committee,

Greetings! When a fire alarm blares, every second counts: a single misstep could send responders into another round of life-changing delay. This is exactly what you and my team are trying to avoid through better planning. Perhaps it's time to look to more reliable, precise, and effective tools. Let our team introduce **"RACE-S"** , the **Responder Action and Clearance Evacuation Solver**, an outstanding solution rooted in mathematical modeling and commitment to saving lives.

I. Our Model. The situation in a burning building is ever-changing. Thanks to our real-time planner **RACE-S**, we are able to solve this problem quite elegantly. To begin with, we simulate a building as a **time-dependent, edge-weighted graph**. Its nodes represent key locations, which can be obtained from a simple blueprint. Complicated factors must be considered: break-in time, room visibility, hazard slowdowns, and a lot more. We even take the presence of pets into consideration.

Next, we work out the best possible sweeping strategy. If an emergency erupts, there's no time to twiddle our thumbs—the optimal **number and location of responders** need to be ready beforehand and we can obtain responder settings from a solver based on greedy room selection. Then we determine **the best sweeping path**, designing a **real-time** planning algorithm that repeatedly optimizes routes as new information is revealed. In this way we can ensure no responders overload and all occupants are safeguarded.

II. Our Recommendations. We have verified our model for two complex buildings. **The hospital inpatient ward** is an excellent example of buildings with special layouts and occupants. Patient rooms account for the majority, while the occupants in question mainly consist of weak patients, perhaps even on life support—they will definitely need assistance. Our optimizer recommends a team of 4 responders: an increase of 1 responder to 4 downsizes the expected time from 1300 s to 300 s, while more responders produce only small improvements but steadily add to the burden. **The multi-use office building** is a more complicated case. Made up of three floors, rooms vary greatly in size and type. The distribution of occupants will be more random. For this situation, RACE-S recommends **6 responders**, achieving a **sixfold** increase in efficiency: a 6.5 minute evacuation in a three-floor building. Both applications also produced an optimal sweeping route, shown clearly in the figure below. Note that our model prioritizes high-probability areas, aiming to perform with efficiency.



RACE-S produces results that are effective, efficient and ready for action. Implementing this strategy into emergency planning will undoubtedly increase secureness and chances of survival. Our team sincerely hopes that the Local Emergency Planning Committee can embrace this strategy and adopt an emergency plan that performs exceptionally when it matters most. Thank you for your attention and dedication on this matter.

Best regards,

HiMCM Team #16843

1 Introduction

1.1 Background

When a fire breaks out in a building, panic often spreads faster than smoke. Such emergencies can occur at any time and may lead to injury or death among occupants who lack mobility or awareness. Over the past 10 years, fire-caused deaths annually in the United States, numbering in the thousands, have increased by 27% [1]. Trained responders therefore play a crucial role in ensuring the safe evacuation of occupants before severe consequences arise.

Evacuation and route planning must be highly efficient, yet multiple factors make it extremely challenging to determine an optimal strategy. Building layouts differ widely in stairwells, number of floors, exit locations, and other structural features. The number and expertise of responders, together with the characteristics of residents—including age, population size, and special needs—must be taken into account. The type of emergency further increases complexity; for example, a gas leak situation poses very different challenges. These factors create significant obstacles for responders sweeping the building and can delay evacuation when time is critical.

Therefore, an effective and efficient mathematical model that determines the best possible sweeping strategy for responders under intricate scenarios is crucial for safe and timely clearance.

1.2 Problem Restatement

In this problem, we must propose a model that generates strategies for safe and efficient sweeping by responders in buildings of varying complexity.

We should begin by identifying key factors that influence room-clearing time, responder movement speed, and additional elements required for different types of emergencies. These factors must remain realistic to support practical use.

Using a fire emergency as an example, we specify the model's objective and design a solver that, given a floor plan, two firefighters' initial positions, and all relevant parameters, outputs optimal sweeping paths for both responders.

The model must also scale to any building layout, any number of responders, and any emergency location, while incorporating responder abilities, emergency types, and unexpected conditions inside rooms.

After evaluating the model, we must interpret the results and estimate which technologies and to what quantitative extent they could improve evacuation safety and efficiency. This analysis will support our recommendation to the Local Emergency Planning Committee and demonstrate that the proposed strategies can contribute to a more secure community.

1.3 Our Work

In this paper, we build a real-time, integrated simulation–optimization framework for emergency sweeps in multi-floor buildings. Our workflow is demonstrated in Fig. 1.1.

- We model a building as a time-dependent, edge-weighted graph. Its node attributes encode room-level service times and occupant dynamics, while its edge weights capture transit times under ever-changing hazards and responder load.
- On this graph representation, we design a real-time planning algorithm that repeatedly optimizes and reallocates responder routes as new information gets revealed. The planner combines a greedy initialization with a local neighborhood search and supports planning for the action of any number of responders under a computational budget.
- Finally, we extend the model to another complex emergency (odorless gas leaks) and investigate the realistic factors we considered. We demonstrate how these extensions affect our model and how we adjust the planning algorithm. We also discuss technologies such as AR glasses and sensors, which are able to

ensure responders' safety and effectively reduce sweep time.

We apply the framework to three layouts: (a) the one-story office with two exits from the contest statement, (b) a two-floor hospital ward, and (c) a multi-use office building with recreational and shared spaces. For each, we estimate the optimal number of responders, their starting positions, and their routes, and analyze the trade-off between sweep time and responder effort.

1.4 Summary of Factors

As a preview, our sweep path planner on the graph is primarily driven by room-level service times and transit costs between rooms and exits. Table 1.1 lists major variables and symbols in this paper for reference. The main influential factors are:

- **Criteria used to determine if a room is all-clear**: We account for area, clutter, and visibility. If obstacles block sight or if non-ambulatory occupants are reported, the room must be fully swept; otherwise, responders can assess it immediately.
- **Occupants within a room**: We distinguish ambulatory and non-ambulatory occupants; pets are treated as non-ambulatory and remain until assisted. Ambulatory occupants may self-evacuate or help non-ambulatory ones evacuate, yet may become non-ambulatory due to smoke inhalation or other factors.
- **Number and type of expertise of responders needed**: For any responder team size, the model assigns positions to minimize total cost while prioritizing rooms with higher non-ambulatory risk. Responders each have expertise that enables them to move faster, guide people more smoothly or even break into a room more easily. In particular, there might be rooms whose inspection demands certain skills that only a minority of responders master.
- **Amount and type of redundancy to incorporate into sweep strategies**: We consider service time in rooms as probability distributions during planning of sweep path and responder settings, covering out-of-distribution rare cases. Our comprehensive consideration ensures a solid foundation for verification.
- **Type of emergency**: For fire, we include smoke spread and fire location. For gas leaks, we incorporate unknown leak sources, higher rates of non-ambulatory occupants, and lower self-evacuation probabilities.

2 Assumptions and Justifications

We consider several assumptions that simplify building layout factors and hazard factors:

1. People moving in corridors do not affect responder speed, assuming that corridors are sufficiently wide.
2. We neglect differences between ascending and descending stair times for relatively short staircases, as the differences are small for this approximate model.
3. Smoke or gas spreads at a constant speed through corridors and stairs.

Also, the unpredictable nature of human behaviors and abilities is simplified:

1. Responders differ only in movement speed; all possess the basic expertise needed for special rooms such as laboratories or nurseries.
2. A room's front door is unlocked only if all occupants have exited unaided or a responder has forced entry. Occupants will not leave the door open and expose others to smoke while people remain inside.
3. Only ambulatory individuals require guiding and comforting; they can, in turn, assist non-ambulatory occupants in the same room.
4. All ambulatory occupants share the same tendency to evacuate at any given moment, so do non-ambulatory ones.

Since sweep time validation requires "ground truth" in reality that is unseen by the planner and is also not provided in the problem, we assume that the ground truth lies in the probabilistic space defined for our model, and follows the prior knowledge we modeled.

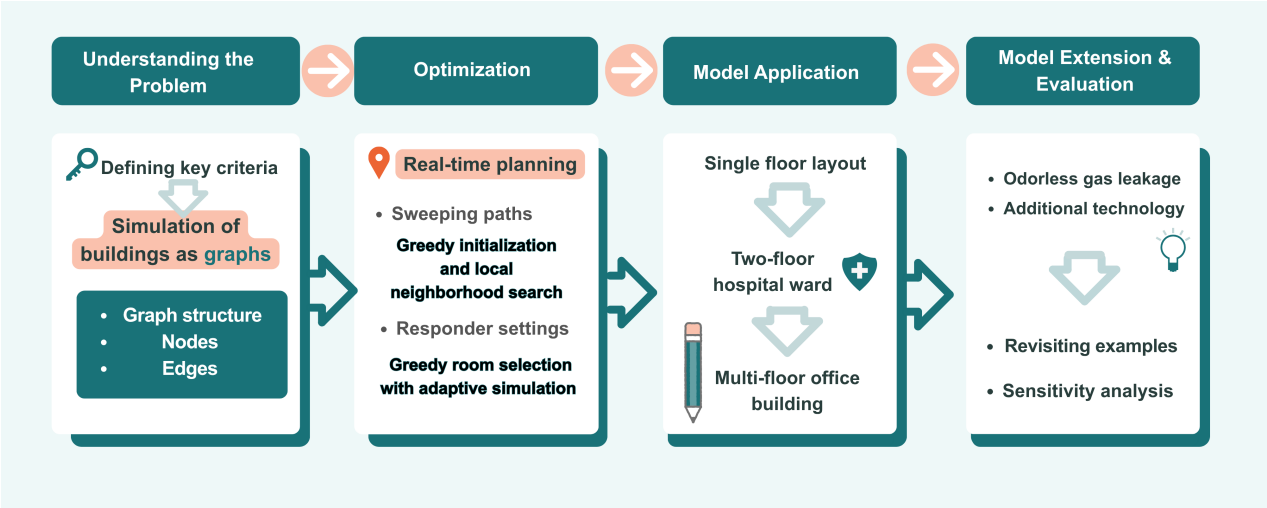


Figure 1.1: Workflow of this paper.

Table 1.1: Table of variables

Symbol	Meaning	Symbol	Meaning	Symbol	Meaning
$M(t)$	ambulatory occupants	$N(t)$	non-ambulatory occupants	$O(t)$	safe/evacuated occupants
T_s	room service time	T_{entry}	entry time	T_{ins}	inspection time
T_{ast}	assistance time	A	room area	c	clutter ratio
c_{fixed}	clutter threshold	d_{room}	inspection walking distance	v	responder speed
t_{knock}	knocking time	t_{break}	forcible-entry time	t_{guide}	guide time per person
t_{calm}	calming time per person	k_{calm}	calming proportion	Δt	inter-event time
$T_w^e(t)$	edge travel time	$T_{w,0}^e$	hazard-free travel time	d_e	edge length
ρ	edge difficulty factor	$S_e(t)$	hazard index on edge	v_{haz}	hazard spread speed
f_{haz}	hazard slowdown factor	f_{load}	load slowdown factor	I_{carry}	carrying indicator
γ_{load}	load penalty parameter	$\lambda_{X\to Y}$	transition rate	$R(t)$	total event rate
G_1	initial graph	G_2	processed graph	G_ω	sample graph
R_{task}	remaining rooms	$\mathcal{P}$	responder route plan	$J(\mathcal{P})$	sweep objective value
D	shortest-time matrix	m	number of responders	S_m	responder start set
J_{resp}	total responder cost	R_{resp}	reward	w	room risk weight

3 Simulating Buildings and Occupants Using Graphs

We abstract the intricate building layout into a graph-based simulator (Section 3.1), where occupant and responder actions are governed by realistic, time-based factors. We define the **prior knowledge** required for sweep path planning in Section 3.2 for rooms (graph nodes) and Section 3.3 for pathways (graph edges).

3.1 Describing a Building with a Graph Structure

We model the building layout as a simple, undirected, edge-weighted graph $G_1 = (V, E)$ with metadata on nodes V and edges E [2] used for sweeping and responder setup. This representation is intentionally agnostic to 2D/3D coordinates, enabling a unified treatment of multi-floor and multi-wing layouts. Fig. 3.1 shows the corresponding graph for the basic scenario in the problem statement.

Node types and properties. Nodes V describe rooms, pathways, and exits. The set V consists of three disjoint subsets:

$$V = R \,\dot\cup\, E_x \,\dot\cup\, T, \tag{3.1}$$

where

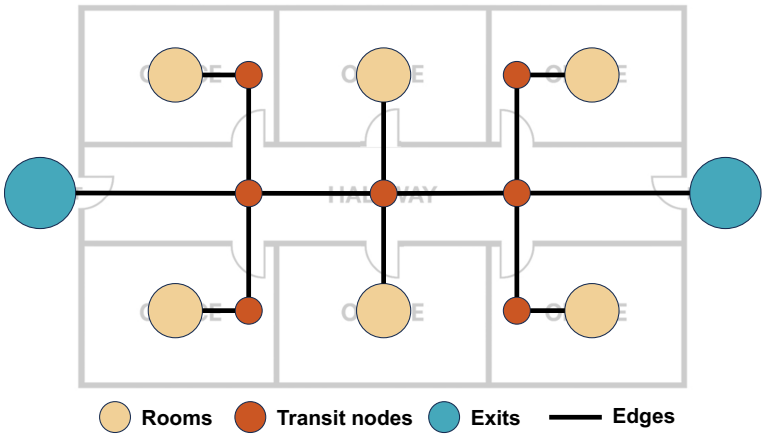


Figure 3.1: Graph representation for the basic scenario.

- R are rooms to be searched, placed at their geometric centers. Each node $i \in R$ carries a service time $T_s^i(t) \geq 0$, determined by factors that vary with time t, especially the number of different occupants;
- E_x are exits where ambulatory occupants leave and non-ambulatory ones are assisted out by responders;
- T are transit nodes (corridor waypoints, intersections, stair landings, etc.) used to compute distances between rooms and exits when Euclidean distance is not appropriate.

Edges and costs. Edges E define pathways between nodes in V and the corresponding transit times. Initially, only adjacent nodes are connected. Each edge $e = (u, v) \in E$ is assigned a time-dependent transition cost $T_w^e(t)$ between nodes u and v to capture dynamic hazards or the responder's load (e.g., carrying occupants).

Hiding information from the planner. The initial graph G_1 contains full information for rooms and pathways, including ground-truth factors that responders should not know before sweeping. As shown in Fig. 3.2, we therefore preprocess G_1 into a planning graph G_2 that conceals room-level details, especially occupant counts, while retaining prior knowledge in the form of probability distributions. Hidden information in G_1 is revealed only after a room is inspected.

We assume that, given these probability distributions, G_1 represents a possible sample from G_2. The following subsections define the prior distributions and other prior knowledge encoded in G_2 and describe their time evolution, which G_1 follows as well. Since G_1 is not provided, for validation purposes, we could sample G_ω from G_2 to represent the synthetic ground truth value.

3.2 Room Nodes R Description

Before defining the room service time T_s, we specify how the number of occupants in a room develops, as this strongly influences T_s. We then define T_s itself, accounting for entry, inspection, and assistance. Finally, each room is assigned a priority weight, elevated for rooms with emergencies, higher likelihood of children, and similar factors.

3.2.1 Number of Occupants in a Room

Initial occupant counts are treated as ground-truth values in G_1 and remain hidden from the planner in G_2 until the room is inspected. During evacuation, the number of occupants present in a room is governed by two factors: **shifts in occupants** and **rescues by responders**. Occupant counts change when occupants take actions at times

$$t_1 = 0 + \Delta t_1,\ t_2 = t_1 + \Delta t_2,\ \cdots,\ t_k = \sum_{i=1}^{k} \Delta t_i,\ \cdots \tag{3.2}$$

or responders take action at times $\tau_1, \tau_2, \cdots, \tau_n$ $(0 < \tau_1 < \tau_2 < \cdots)$.

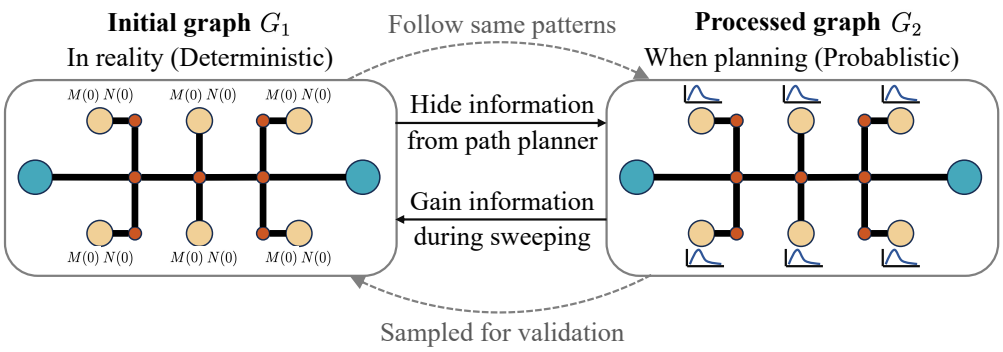


Figure 3.2: The relation between initial graph G_1 and processed graph G_2.

We classify three occupant types as: (a) **ambulatory** occupants present in the room; (b) **non-ambulatory** occupants present (pets included); and (c) **safe** occupants, having already exited. For time $t \geq 0$ (evacuation start), we define

- $M(t)$ as the number of ambulatory occupants;
- $N(t)$ as the number of non-ambulatory occupants, including pets;
- $O(t)$ as the number of safe occupants.

True values $M(0)$ and $N(0)$ in G_1 remain unexposed to G_2, whose prior distributions specify probabilities over possible $M(0)$ and $N(0)$ values that differ by room type in Table 3.1. M and N then develop through occupant and responder events defined below and will determine the time required to sweep each room.

Shift in occupants

In this part, we focus on occupant actions resulting in changes of room status. The time-dependent evolution of $(M(t), N(t), O(t))$ is driven by three occupant event types, which may occur at any time t:

1. $M \to N$: An ambulatory occupant becomes non-ambulatory, i.e., $(M, N, O) \to (M-1, N+1, O)$;
2. $M \to O$: An ambulatory occupant self-evacuates, i.e. $(M, N, O) \to (M-1, N, O+1)$;
3. $N \to O$: A non-ambulatory occupant evacuates with peer occupants' assistance, i.e. $(M, N, O) \to (M, N-1, O+1)$.

We simulate occupant dynamics by describing these transitions as probabilistic events over time, analogous to a Gillespie algorithm [3] for chemical reaction simulation. Each transition is assigned a time-dependent *per-person instantaneous rate* $\lambda_{M\to N}(t)$, $\lambda_{M\to O}(t)$, and $\lambda^{\text{peer}}_{N\to O}(t)$, such that for a small interval $[t, t+h)$ the probability of a given transition is approximately $\lambda(t)\, h$. We set

$$\lambda_{M\to N}(t) = \tanh(c_1 t), \quad \lambda_{M\to O}(t) = \tanh(c_2 t), \quad \lambda^{\text{peer}}_{N\to O}(t) = \tanh(c_3 t), \tag{3.3}$$

where c_1 depends on distance from the emergency location and emergency type, while c_2 and c_3 are $O(10^{-3})$ constants determined by emergency type and information communication efficiency. As shown in Fig. 3.3, each $\tanh(ct)$ increases monotonically and more slowly over time, reflecting that both the risk of becoming non-ambulatory and the likelihood of self-evacuation evolve as the incident unfolds.

For simulation, we approximate these smoothly varying rates by *piecewise-constant* functions on short, manually chosen time intervals. With a macro time step $\Delta T > 0$, we partition time into

$$I_k = [k\Delta T, (k+1)\Delta T), \qquad k = 0, 1, 2, \ldots, \tag{3.4}$$

with left endpoints $t_k = k\Delta T$. On each interval I_k, we freeze the per-person rates at

$$\lambda_{X\to Y}(t) \approx \lambda^{(k)}_{X\to Y} := \lambda_{X\to Y}(t_k), \quad t \in I_k. \tag{3.5}$$

Table 3.1: Probabilities of $M(0)$ and $N(0)$ values in different rooms.

Room type	$(M(0), N(0))$	Prob.	Room type	$(M(0), N(0))$	Prob.	Room type	$(M(0), N(0))$	Prob.
Single office	$(1,0)$	0.8	Restroom	$(2,0)$	0.6	Medium office	$(3,1)$	0.3
	$(2,0)$	0.1		$(4,0)$	0.2		$(5,0)$	0.4
	$(3,0)$	0.1		$(6,0)$	0.2		$(8,0)$	0.3
				$(1,1)$	0.1			
Shared office	$(1,0)$	0.1	Nurse station	$(0,1)$	0.4	Hospital equipment room	$(0,1)$	0.5
	$(5,0)$	0.4		$(1,1)$	0.4		$(1,1)$	0.4
	$(10,0)$	0.3	Hospital patient room	$(1,1)$	0.4	Storeroom	$(0,0)$	0.8
	$(15,0)$	0.2		$(2,1)$	0.3		$(1,0)$	0.15
				$(3,1)$	0.2		$(2,0)$	0.05
Meeting room	$(8,0)$	0.5	Cafe	$(3,0)$	0.4	Recreation room	$(5,0)$	0.4
	$(12,0)$	0.3		$(7,0)$	0.3		$(10,0)$	0.3
	$(20,0)$	0.2		$(10,0)$	0.2		$(15,0)$	0.2

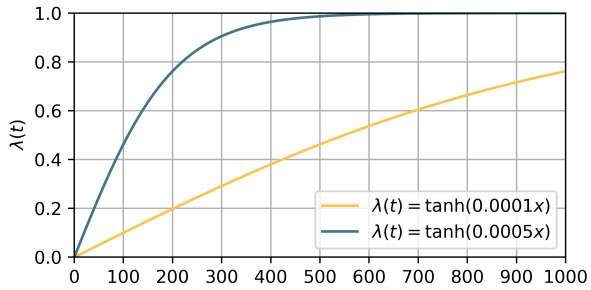


Figure 3.3: Two $\tanh$ functions with varying c values.

Given current time t and state $(M(t), N(t), O(t))$ in I_k, the instantaneous event rates are

$$r_{M\to N}(t) = \lambda^{(k)}_{M\to N}\, M(t), \quad r_{M\to O}(t) = \lambda^{(k)}_{M\to O}\, M(t), \quad r^{\text{peer}}_{N\to O}(t) = \lambda^{\text{peer},(k)}_{N\to O}\, N(t), \tag{3.6}$$

and the total rate is

$$R(t) = r_{M\to N}(t) + r_{M\to O}(t) + r^{\text{peer}}_{N\to O}(t). \tag{3.7}$$

Between successive events the state (M, N, O) is constant, so with the piecewise-constant approximation the rates remain fixed until the next event or the end of the interval. We then draw a random waiting time

$$\Delta t_i \sim \text{Exp}\big(R(t)\big), \qquad \Delta t_i = -\frac{1}{R(t)} \ln U,\ U \sim \text{Uniform}(0,1), \tag{3.8}$$

using the rates for the current interval. Fig. 3.4 shows the probability distribution functions (PDFs) of Δt_i for various $R(t)$.

After drawing the random Δt_i, if $t + \Delta t_i < (k+1)\Delta T$, an event is chosen and executed at $t + \Delta t_i$, and we update the clock to $t + \Delta t_i$ accordingly. Since rates are constant on $(t, t + \Delta t_i)$, events at t and at $t + \Delta t_i$ follow the same mechanism. If $t + \Delta t_i \geq (k+1)\Delta T$, we advance the clock to $t_{k+1} = (k+1)\Delta T$ without firing an event, update rates for the next interval, and continue. We then update (M, N, O) and repeat until the room is empty. The executed event type is chosen with probabilities:

$$p_{M\to N} = r_{M\to N}(t)/R(t), \quad p_{M\to O} = r_{M\to O}(t)/R(t), \quad p_{N\to O} = r^{\text{peer}}_{N\to O}(t)/R(t). \tag{3.9}$$

Rescue by responders

Responder actions also alter room dynamics by reducing the number of occupants through assisted evacuation. We assume responders act on a room at times $\tau_1, \tau_2, \ldots$, where the τ_k are determined by the path planner rather than specified as in occupant-driven events. At each τ_k, the responder system prescribes one of two actions:

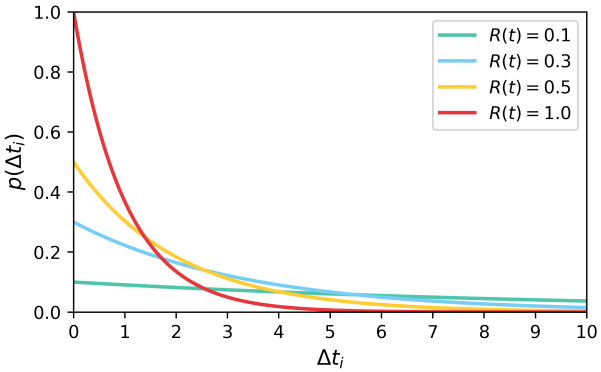


Figure 3.4: Probability distribution functions of Δt_i given four values of $R(t)$.

- $(M, N, O) \to (M, N-1, O+1)$ when a non-ambulatory occupant is carried out, or
- $(M, N, O) \to (M-1, N, O+1)$ when an ambulatory occupant is escorted from the room.

Responder-driven movements occur at times $\tau_1, \tau_2, \ldots$ and are treated as an external decision process planned separately. These actions jointly update room occupancy with the occupant-driven events at times $t_1, t_2, \ldots$ (Eq. (3.2)). After each responder intervention, we recompute occupant-driven transitions since the occupant counts have changed. Between all discrete event times, the state $(M(t), N(t), O(t))$ remains constant.

3.2.2 Service Time T_s^i Spent on a Room i

We divide room service time into three components: **entry**, **inspection**, and **assistance**. Entry means using brute force to open the door when it is locked. Inspection includes traversing the room. Assistance concerns calming panicked individuals and accompanying non-ambulatory occupants (initially disabled or incapacitated during the incident) to an exit, and returning to the room. Let the total time

$$T_s^i(t; \boldsymbol{p}_i) = T_{\text{entry}} + T_{\text{ins}} + T_{\text{ast}}, \tag{3.10}$$

where T_{entry} refers to time spent on entry, T_{ins} is time for inspection, T_{ast} denotes the time spent on assistance to evacuation, and the vector $\boldsymbol{p}_i$ collects probabilistic parameters like the prior distributions of $M(0)$ and $N(0)$.

Entry begins with knocks; if unanswered, responders turn to forced entry. A door may be locked ($I_{\text{lock}} = 1$) or unlocked ($I_{\text{lock}} = 0$). If unlocked, responders can immediately understand whether the room is vacant or if non-ambulatory occupants remain (as we assume that occupants leave the door open when fully evacuated). If the door is **locked**, the responder **knocks three times** until someone opens it or a time threshold is reached.

We model knock time as

$$t_{\text{knock}} \sim \Gamma(3, \alpha + \beta\lambda_{\text{knock}}), \tag{3.11}$$

where Γ stands for the Γ-distribution [10][1], $\alpha, \beta > 0$ are global parameters, and λ_{knock} depends on the number of ambulatory occupants (Section 3.2.1). More ambulatory people increase the chance someone opens the door, therefore increasing λ_{knock}. If $t_{\text{knock}} > 10\,\text{s}$, the responder begins **forced entry**, taking $t_{\text{break}} = 15\,\text{s}$. Therefore, we have

$$T_{\text{entry}} = \begin{cases} t_{\text{knock}}, & t_{\text{knock}} \le 10\ \text{s}, \\ 10\ \text{s} + t_{\text{break}}, & t_{\text{knock}} > 10\ \text{s}. \end{cases} \tag{3.12}$$

Inspection time quantifies how long a responder needs to obtain full information about the room, i.e., how many occupants remain, depending on area, clutter, and the distance walked during inspection. We define:

- v: the moving speed of the responder,

[1] In this paper, the parameter β of $\Gamma(\alpha, \beta)$ is a rate parameter.

- c: the clutter factor, defined as $c = A_{\text{obs}}/A$, where A is room area and A_{obs} is the area covered by obstacles,
- c_{fixed}: a predefined clutter threshold above which inspection is required,
- $I_{\text{non-amb}}$: indicator of non-ambulatory occupants (1 if present, 0 otherwise),
- d_{room}: approximate walking distance, taken as the room perimeter.

If obstacles do not block visibility ($c \le c_{\text{fixed}}$), the responder immediately assesses the room and, if found non-ambulatory occupants ($I_{\text{non-amb}} = 1$), can move directly to the non-ambulatory occupant. If the room is cluttered or non-ambulatory occupants are known to be inside, a full sweep is required, with time depending on the perimeter d_{room}, area A, and the degree of clutter. Thus, we define

$$T_{\text{ins}} = \begin{cases} 0 & \text{if } c \le c_{\text{fixed}} \text{ and } I_{\text{non-amb}} = 0 \\ \frac{d_{\text{room}}}{v} & \text{if } c \le c_{\text{fixed}} \text{ and } I_{\text{non-amb}} = 1 \\ \frac{d_{\text{room}}}{v}(1+c) + \mu A\left(1 + \frac{c}{2}\right) & \text{if } c > c_{\text{fixed}} \end{cases} \tag{3.13}$$

where μ scales the contribution of room size and clutter.

Assistance covers guiding and comforting ambulatory occupants and carrying out non-ambulatory ones. We introduce:

- t_{guide}: time to guide each person, including travel to and from the exit, considering all factors described in the next subsection,
- k_{calm}: proportion of occupants requiring calming (typically higher when children are present),
- t_{calm}: time spent calming each such occupant.

These are global parameters determined by building and emergency type. With $M(t)$ the number of ambulatory occupants when the responder arrives (Section 3.2.1), we define

$$T_{\text{ast}} = M(t)t_{\text{guide}} + M(t)k_{\text{calm}}t_{\text{calm}}. \tag{3.14}$$

3.2.3 Risk Weight of a Room

To prioritize vulnerable occupants and hazardous spaces, each room $i \in R$ is assigned a nonnegative risk weight w_i. Rooms likely to contain children, elderly, or patients, or located near hazards, should be swept earlier than low-use spaces. We decompose risk into three parts:

- **Occupant vulnerability** $z^i_{\text{vul}} \in \{0, 1, 2\}$: 0 for healthy adults (offices), 1 for mixed populations (meeting rooms), and 2 for vulnerable groups (patient rooms and others).
- **Expected occupancy** $z^i_{\text{occ}} \ge 0$: proportional to $\mathbb{E}[M_i(0) + N_i(0)]$ (Table 3.1), normalized by a reference capacity C_{ref}:
$$z^i_{\text{occ}} = \mathbb{E}[M_i(0) + N_i(0)]/C_{\text{ref}}. \tag{3.15}$$
- **Hazard proximity** $z^i_{\text{haz}} \in \{0, 1\}$: 1 if room i is on the same floor and within graph distance d_{haz} of the emergency origin s, and 0 otherwise.

The final weight is a linear combination,

$$w_i = 1 + \alpha_{\text{vul}} z^i_{\text{vul}} + \alpha_{\text{occ}} z^i_{\text{occ}} + \alpha_{\text{haz}} z^i_{\text{haz}}, \tag{3.16}$$

with $\alpha_{\text{vul}}, \alpha_{\text{occ}}, \alpha_{\text{haz}} \ge 0$ chosen so that w_i typically lies between 1 (low priority) and about 5 (critical). In our simulations, we set $\alpha_{\text{vul}} = 1.5$, $\alpha_{\text{occ}} = 1.0$, and $\alpha_{\text{haz}} = 1.0$, giving highest weights to patient rooms or other vulnerable rooms near the hazard.

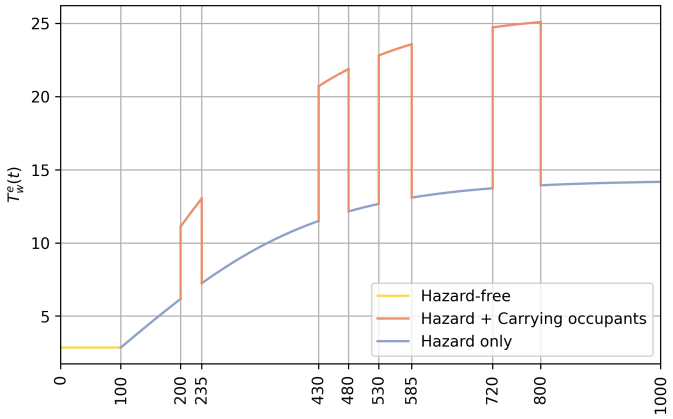


Figure 3.5: Transit time $T_w^e(t)$ under hazard growth and responder load.

3.3 Edges E Description: Transit Time T_w^e Spent on an Edge e

Room nodes R model in-room processes, while edges E represent corridor and stair segments connecting nodes. For planning, each edge $e = (u, v) \in E$ has a time-dependent transit cost $T_w^e(t)$, the time a responder needs to traverse it starting at global time t. Transit time is assumed symmetric in direction and unaffected by corridor congestion; only hazards and responder load (carrying a person) cause slowdowns.

We calculate $T_w^e(t)$ in the following way:

$$T_w^e(t) = T_{w,0}^e \, f_{\text{haz}}^e(t) \, f_{\text{load}}^r(t). \tag{3.17}$$

Hazard-free travel time $T_{w,0}^e$. Each edge has physical length $d_e = d(i, j) > 0$ between two nodes $i, j \in V$, and is either corridor or stair. A responder's base walk speed is v (without slowdown factors), and each edge type has difficulty factor $\rho_e \geq 1$ ($\rho = 1$ for corridors and $\rho = 1.5$ for stairs). Thus,

$$T_{w,0}^e = \rho \, d_e / v. \tag{3.18}$$

Time-varying hazard slowdown $f_{\text{haz}}^e(t)$. Smoke, gas, or heat along passages is modeled through a hazard index $S_e(t) \in [0, 1]$ on the edge that is 1 when hazard reaches the edge. Let s be the hazard's origin node and $d_s = \min(d(s, i), d(s, j))$ the distance to endpoints of the edge. Given a hazard speed v_{haz}, the time at which the hazard reaches edge e is $t_e^{\text{arr}} = d_s / v_{\text{haz}}$. For $t \geq t_e^{\text{arr}}$, $S_e(t)$ increases following

$$S_e(t) = \begin{cases} 0, & t < t_e^{\text{arr}}, \\ \tanh\big(\sigma(t - t_e^{\text{arr}})\big), & t \geq t_e^{\text{arr}}. \end{cases} \tag{3.19}$$

where $\sigma = 0.003$ is a scale factor. The resulting hazard multiplier is:

$$f_{\text{haz}}^e(t) = 1 + \beta_{\text{haz}} S_e(t) \tag{3.20}$$

where $\beta_{\text{haz}} \geq 0$ and large values of $f_{\text{haz}}^e(t)$ effectively implies that an edge is impassable.

Load-dependent slowdown $f_{\text{load}}(t)$. If responder r finds a non-ambulatory occupant, they accompany or carry that occupant to the exit and then return to the same room. Such load affects all subsequent edges until an exit is reached. Let $I_{\text{carry}} \in \{0, 1\}$ indicate load status. Then

$$f_{\text{load}}(t) = 1 + \gamma_{\text{load}} \, I_{\text{carry}}, \tag{3.21}$$

where $\gamma_{\text{load}} > 0$ is a constant parameter.

These edge costs, combined with room service times T_s^i from Section 3.2, determine the full sweep path cost. Fig. 3.5 illustrates the evolution of $T_w^e(t)$: initially without hazard, increasing as hazard such as smoke arrives, and exhibiting sharp jumps when the responder carries non-ambulatory occupants.

4 RACE-S: Real-time Planner for Sweeping Path and Responder Setup

Given G_2, which encodes the building layout and prior properties of nodes and edges, we construct a planner, namely Responder Action and Clearance Evacuation Solver (RACE-S), that serves two purposes: (a) During an evacuation, whenever new information is revealed from G_1, it **searches for optimal paths for multiple responders** using observed data and prior knowledge to (approximately) minimize sweep time (Section 4.1, Fig. 4.1). (b) At emergency onset or in advance, it **optimizes responder locations** to balance shortest sweep time and efficient evacuation of high-risk rooms under uncertainty of G_1 that leads to different realized sweep paths (Section 4.2, Fig. 4.2).

4.1 Optimization for Sweeping Paths

4.1.1 Objective

At any time τ, the planner sees only prior room–service distributions in G_2 (true values in G_1 appear only upon inspection). We define:

- $R_{\text{task}} \subseteq R$: rooms still requiring sweep,
- $s_k \in V$: current location of responder $k = 1, \ldots, m$, with the responder's next available time $t_k \geq \tau$.

A *sweeping plan* assigns each $i \in R_{\text{task}}$ to exactly one responder and specifies visit order. A route of length ℓ_k for responder k is

$$\mathcal{P}_k = [\, s_k, i_{k,1}, i_{k,2}, \ldots, i_{k,\ell_k} \,], \quad i_{k,j} \in R_{\text{task}}, \tag{4.1}$$

covering all rooms exactly once across $\{\mathcal{P}_1, \ldots, \mathcal{P}_m\}$. Given a candidate plan $\mathcal{P}$, we simulate responders under prior T_w^e and T_s^i to obtain:

- $T_k(\mathcal{P})$: estimated completion time of responder k,
- $C_i(\mathcal{P})$: estimated clearance time of room i.

Following the contest statement, we minimize overall sweep time (makespan) while favoring early clearance of high-risk rooms. We use the composite objective function

$$\text{minimize}_{\mathcal{P}}\ J(\mathcal{P}) \ = \ \lambda_1 \max_{k=1}^{m} T_k(\mathcal{P}) \ + \ \lambda_2 \sum_{i \in R_{\text{task}}} w_i\, C_i(\mathcal{P}), \tag{4.2}$$

with $\lambda_1 \gg \lambda_2 \geq 0$, ensuring makespan dominates while still rewarding high-priority rooms.

4.1.2 Solver Based on Greedy Initialization and Local Neighborhood Search

We propose an effective sweep path solver, DynamicPlanner, shown in Algorithm 1 and detailed below.

Step 1: Initialize Responder Configuration

A responder configuration is defined by

$$(m, S_m), \quad S_m = \{s_1, \ldots, s_m\} \subseteq R, \quad |S_m| = m, \tag{4.3}$$

where m is the number of responders and S_m is the set of distinct room nodes as starting positions. Here we assume responders start in room nodes and (m, S_m) is given. If entering from exits, they immediately proceed to their assigned start node.

Step 2: Greedy-algorithm-based Initial Plan

Initialization matters for an optimization algorithm. To construct an initial sweeping plan given R_{task} and the current responder states, we propose a simple greedy algorithm, denoted as PolicyGreedyInitialization:

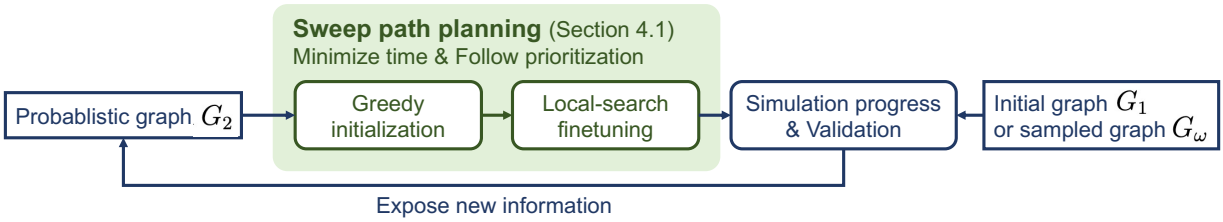


Figure 4.1: Sweep path planner pipeline in Section 4.1

Algorithm 1 DynamicPlanner

Require: Responder count m; start rooms S_m;

(Step 1) Initialize responders at $s_k \in S_m$ with $t_k \leftarrow 0$
while not all rooms are confirmed clear **do**
    Construct current graph $G_2(t)$ based on known and prior information
    Determine current task set $R_{\text{task}}(t)$
    Read current responder states $(s_k(t), t_k)$
    (Step 2-3) $(\mathcal{P}, J_{\text{plan}}) \leftarrow$ SweepSolver$(G_2, R_{\text{task}}, \{(s_k(t), t_k)\}_{k=1}^m)$
    (Step 4) Execute routes $\mathcal{P}$ until the next information update from G_1 or all rooms are cleared
    Update t, building state, $R_{\text{task}}(t)$, and responder states $(s_k(t), t_k)$
end while

- Select the responder who becomes available first.
- For that responder at node s, score each room $i \in R_{\text{task}}$ using the shortest travel time T_{travel} between nodes i and s, expected service time $\mathbb{E}[T_s^i(t + T_{\text{travel}}; \boldsymbol{p}_i)]$, and risk weight.

$$\text{score}(i|s,t) = T_{\text{travel}} + \mathbb{E}[T_s^i(t + T_{\text{travel}}; \boldsymbol{p}_i)]. \tag{4.4}$$

- Assign the room with the minimal score as their next visit.

$$v_{\text{next}} = \arg\min_{i \in R_{\text{task}}} \text{score}(i|s,t). \tag{4.5}$$

The shortest travel time T_{travel} between two nodes is dynamically computed using a time-dependent variation of Dijkstra's algorithm [5], denoted as the function ShortestTravelTime(G_2, s, i, t) but omitted for conciseness. Expectations are computed via Monte Carlo sampling from prior distributions. Updated R_{task} or responder states at different time naturally alter the plan, which makes the planner real-time.

Step 3: Local-search Optimization

The greedy plan is feasible but not optimal. We refine it via a local search strategy evaluating $J(\mathcal{P})$ (Eq. (4.2), denoted as a function EvaluateRoutes$(\mathcal{P})$) repeatedly with two standard routing modifications, which widely exist in vehicle-routing literature [11]:

- **Intra-route 2-opt**: choose $1 \leq a < b \leq \ell_k$ in route $\mathcal{P}_k$ and reverse $i_{k,a} \ldots i_{k,b}$.
- **Inter-route relocate.** remove a room from one route and insert it at all possible positions in another responder's route, retaining the best insertion.

Modifications that strictly reduce $J(\mathcal{P})$ are accepted. Iteration continues until no improvement is found.

Step 4: Updating Sweep Paths with New Information

The overall sweep path planner SweepSolver (Algorithm 3) combines greedy initialization PolicyGreedyInitialization and local search LocalSearchOptimize. It is called at emergency onset and whenever new information inside a room arrives, which update R_{task} and responder states that can change sweep plan.

Algorithm 2 LocalSearchOptimize: improve greedy initialization by recursive modifications

Require: Graph G_2; rooms R_{task}; responder starts (s_k, t_k); initial routes $\mathcal{P}$; max iterations $I_{\max}$
Ensure: Improved routes $\mathcal{P}$ and cost J

$J \leftarrow \text{EvaluateRoutes}(\mathcal{P})$
$i \leftarrow 0$
repeat
  $i \leftarrow i + 1$
  Try a random Intra-route 2-opt modification to get $J_1, \mathcal{P}_1$
  Try a random Intra-route relocate modification to get $J_2, \mathcal{P}_2$
  if J_1 or J_2 is lower than J **then**
    Update J as the lower of J_1 and J_2, and record the corresponding routes to $\mathcal{P}$
  end if
until J is not updated or $i \geq I_{\max}$
return $\mathcal{P}, J$

Algorithm 3 SweepSolver

Require: Current graph G_2; rooms R_{task}; responder states (s_k, t_k)
Ensure: Final routes $\mathcal{P}$ and cost J

$\mathcal{P} \leftarrow \text{PolicyGreedyInitialization}(G_2, R_{\text{task}}, \{(s_k, t_k)\}, \{w_i\}, \{T_s^i\})$
$\mathcal{P}, J \leftarrow \text{LocalSearchOptimize}(G_2, R_{\text{task}}, \{(s_k, t_k)\}, \mathcal{P})$
return $\mathcal{P}, J$

4.2 Optimization for Responder Setting

When an emergency begins or even beforehand, we must determine the optimal number of responders and their starting locations to enable the fastest sweep. This requires accounting for stochastic nature in the true building state G_1 (particularly initial occupants), and achieving strong average performance across random realizations of G_1 sampled from G_2, denoted as G_ω. Thus, we formulate an upper-level optimization layer built on top of the sweeping-path planner in Section 4.1, providing it G_ω that replaces G_1 for simulation validation.

4.2.1 Objective

Given (m, S_m) and a randomly sampled building state G_ω from G_2 (analogous to a ground-truth G_1), we extract the evolving graph $G_2(t; G_\omega)$, task room set $R_{\text{task}}(t; G_\omega)$, and responder states $(s_k(t), t_k)$ during the sweep from Section 4.1. Running the planner until all rooms are cleared yields the random sweep cost

$$J_{\text{real}}(m, S_m; G_\omega) = J(\mathcal{P}) \;=\; \lambda_1 \max_{k=1}^{m} T_k(\mathcal{P}) \;+\; \lambda_2 \sum_{i \in R_{\text{task}}} w_i \, C_i(\mathcal{P}) \quad \text{from Eq. (4.2).} \tag{4.6}$$

Our upper-level objective evaluates the average performance of configuration (m, S_m) across random building states while penalizing excessive responder use:

$$\text{minimize}_{(m,S_m)} \widehat{J}_{\text{resp}}(m, S_m) = \mathbb{E}_{G_\omega}\big[J_{\text{resp}}\big] = \mathbb{E}_{G_\omega}\big[J_{\text{real}}(m, S_m; G_\omega)\big] + c_{\text{resp}}\, m, \tag{4.7}$$

where $c_{\text{resp}} \geq 0$ represents labor cost or responder scarcity and the expectation of J_{resp} is estimated via adaptive Monte Carlo sampling of G_ω from G_2. This formulation automatically detects the point of diminishing returns where adding a responder costs more than the time saved.

4.2.2 Solver Based on Greedy Room Selection with Adaptive Simulation

Step 1: Initializing Start Rooms for a Fixed Number of Responders

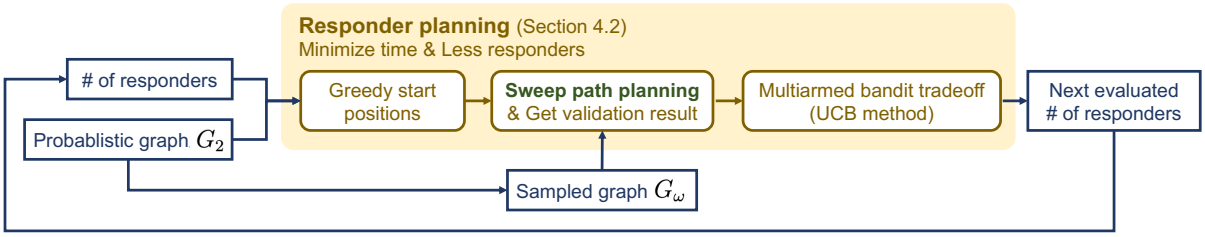


Figure 4.2: Responder configuration planner pipeline in Section 4.2, which calls the sweep path planner in Section 4.1 (Fig. 4.1).

Algorithm 4 SELECTSTARTROOMS

Require: Room set R; pairwise shortest travel time matrix D; risk weights w_i; responder count m
Ensure: Starting-room set $S_m \subseteq R$ with $|S_m| = m$

$S_m \leftarrow \emptyset$
$s_1 \leftarrow \arg\min_{r \in R} \sum_{i \in R} w_i D_{ri}$
$S_m \leftarrow S_m \cup \{s_1\}$
for $\ell \leftarrow 2$ to m **do**
  $s_\ell \leftarrow \arg\min_{r \in R \setminus S_m} \sum_{i \in R} w_i \min\left\{D_{ir}, \min_{s \in S_m} D_{is}\right\}$
  $S_m \leftarrow S_m \cup \{s_\ell\}$
end for
return S_m

For a given responder count m and graph G_2, we must select a good starting set S_m. We use the static travel-time matrix D, where $D_{ij} = \text{SHORTESTTRAVELTIME}(G_2, i, j, t)$ from Section 4.1, together with room risk weights w_i. Responders should start near high-risk rooms while remaining spatially dispersed. We cast this as a risk-weighted problem and solve it via the greedy method detailed in Algorithm 4.

The first responder position r minimizes the total travel time to all other nodes weighted by risk weights w_i for each room i, by iterating on all nodes:

$$\text{minimize}_{r \in R}\ J_{\text{1st-resp}}(r) = \sum_{i \in R} w_i D_{ri}. \tag{4.8}$$

Subsequent responders should reduce the weighted time to reach each room. For any room i, its current closest responder has distance $\min_{s \in S_m} D_{is}$. A new start room r should minimize

$$\text{minimize}_{r \in R \setminus S_m}\ J_{\text{new-resp}}(r) = \sum_{i \in R} w_i \min\left\{D_{ir}, \min_{s \in S_m} D_{is}\right\}. \tag{4.9}$$

Step 2: Simulation-based Evaluation of a Given Responders Count

To optimize the number of responders, we approximate the expectation in Eq. (4.7). For each possible number of responders m, we perform multiple Monte–Carlo trials following two steps: (a) selecting optimal start rooms S_m using Step 1 for the specific realization; (b) sampling G_ω (replacing the true hidden graph G_1) from G_2 and running a full sweep simulation using DYNAMICPLANNER (Algorithm 1). The procedure is described in Algorithm 5.

A single call to $\text{SIMULATEONECONFIGURATION}(m, S_m, \cdot)$ yields a random realized sweep cost $J_{\text{real}}(m, S_m; G_\omega)$. We define the corresponding total cost J_{resp} and reward R_{resp} as

$$J_{\text{resp}}(m, S_m; G_\omega) = -R_{\text{resp}}(m, S_m; G_\omega) = J_{\text{real}}(m, S_m; G_\omega) + c_{\text{resp}} m. \tag{4.10}$$

Algorithm 5 SIMULATEONECONFIGURATION (one Monte–Carlo run)

Require: Responder count m; start rooms S_m; random seed or scenario id used to sample G_ω
Ensure: Realized sweep cost $J_{\text{real}}(m, S_m; G_\omega)$
1: Sample world randomness G_ω from G_2 (occupants, hazards, blockages, etc.)
2: Initialize building state from G_ω
3: Execute DYNAMICPLANNER
4: Compute realized cost $J_{\text{real}}(m, S_m; G_\omega)$ from the simulation log
5: **return** $J_{\text{real}}(m, S_m; G_\omega)$

Algorithm 6 UCBRESPONDERCOUNT (adaptive choice of responder number)

Require: Possible responder numbers $\mathcal{M}$; start rooms set S_m; total simulations $N_{\max}$; per-responder cost c_{resp}
Ensure: Recommended responder count $m^\star$
1: **for all** $m \in \mathcal{M}$ **do**
2: $n_m \leftarrow 0$, $\bar{R}^m_{\text{resp}} \leftarrow 0$, $s^m_{\text{resp}} \leftarrow 0$
3: **end for**
4: **for** $i \leftarrow 1$ to $N_{\max}$ **do**
5: $m_i \leftarrow \arg\max_{m \in \mathcal{M}} \text{UCB}_m$
6: $J^m_{\text{real}} \leftarrow$ SIMULATEONECONFIGURATION(m_i, S_{m_i})
7: $J^m_{\text{resp}} \leftarrow J^m_{\text{real}} + c_{\text{resp}} m_t$; $R^m_{\text{resp}} \leftarrow -J^m_{\text{resp}}$
8: Update $(n_m, \bar{R}^{m_i}_{\text{resp}}, s^{m_i}_{\text{resp}})$ with reward R^m_{resp}
9: **end for**
10: $m^\star \leftarrow \arg\max_{m \in \mathcal{M}} \text{LCB}_m$ ▷ Most conservative high-reward (low-cost) arm
11: **return** $m^\star$

For each $m \in \mathcal{M}$, after n_m simulations we track the empirical mean reward $\bar{R}^m_{\text{resp}}$ and sample standard deviation $s^m_{\text{resp}} = \text{Std}[R^m_{\text{resp}}]$.

Estimating Eq. (4.7) via exhaustive simulation in the large probabilistic space in G_2 is infeasible, so we adopt an adaptive m allocation strategy inspired by the UCB policy from Auer et al. [4] (Algorithm 6). Each responder count $m \in \mathcal{M} = \{m_{\min}, \ldots, m_{\max}\}$ with its start-room set S_m (from Algorithm 4) is treated as a single arm in a multi-armed bandit problem [6]. Our goal is to identify the m with the highest expected reward $\mathbb{E}[R^m_{\text{resp}}]$, approximated by $\bar{R}^m_{\text{resp}}$, under a total budget of $N_{\max}$ simulations. We set $N_{\max} \geq 2|\mathcal{M}|$ so that each arm is sampled at least twice and s^m_{resp} is well defined.

At iteration t, the selected arm m maximizes

$$\text{UCB}_m \;=\; \bar{R}^m_{\text{resp}} + t_{1-\delta_1/2,\, n_m-1}\, s^m_{\text{resp}} \sqrt{\frac{2 \ln t}{n_m}}, \tag{4.11}$$

where $t_{1-\delta_1/2,\, n_m-1}$ is the $(1 - \delta_1/2)$ Student-t quantile [9] and n_m the number of samples for arm m.[2]

For the final decision, we choose the responder count $m^\star$ maximizing the $(1 - \delta_2)$ lower confidence bound

$$\text{LCB}_m \;=\; \bar{R}^m_{\text{resp}} - t_{1-\delta_2/2,\, n_m-1}\, \frac{s^m_{\text{resp}}}{\sqrt{n_m}}. \tag{4.12}$$

This adaptive bandit strategy concentrates simulations on promising responder counts while still exploring uncertain options of m. The final choice $m^\star$ is the smallest number of responders whose total sweep cost is consistently low across stochastic building realizations G_ω.

[2]When $n_m < 2$ and s^m_{resp} is undefined, we define $\text{UCB}_m = +\infty$ to prioritize exploration of these arms.

5 Results of Responders Reacting to a Fire Alarm

We apply our model to three building layouts of varying size and floor structure, performing sensitivity analysis across random seeds to demonstrate robustness.

For each scenario, we visualize sweeping paths and responder configurations on the floor plans. Each responder is assigned a unique color; circles mark starting positions and arrows indicate final locations. Paths appearing on multiple floors correspond to responders moving between levels via stair edges while completing their assigned sweeps. For the following three major results, parameters of our sampled G_ω and sample outputs are briefly shown in Appendix.

5.1 Layout 1: The Provided Floor Plan of an Office

As a proof of concept, we apply our sweep-path planner to the contest's office layout (Fig. 5.1) using a fixed responder count $m = 2$ and one sampled building state G_ω (treated as G_1). After applying our model, we are able to obtain the total sweeping time, **77.1 seconds**, with the first responder sweeping for 69.6 seconds and the second responder sweeping for 77.1 seconds. The resulting routes are shown in Fig. 5.2. The floor is cut in half, with each responder clearing 3 rooms in order. This plan matches human intuition as to travel in indentical routes in symmetric layouts.



Figure 5.1: The provided office layout. Designed using an online tool on `www.homestyler.com`.

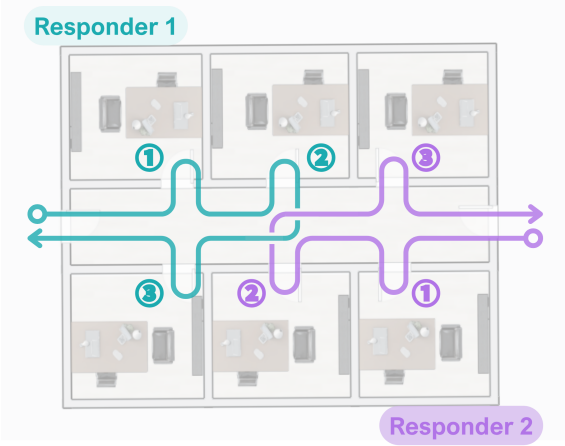


Figure 5.2: Planned sweep path for two firefighters in Layout 1.

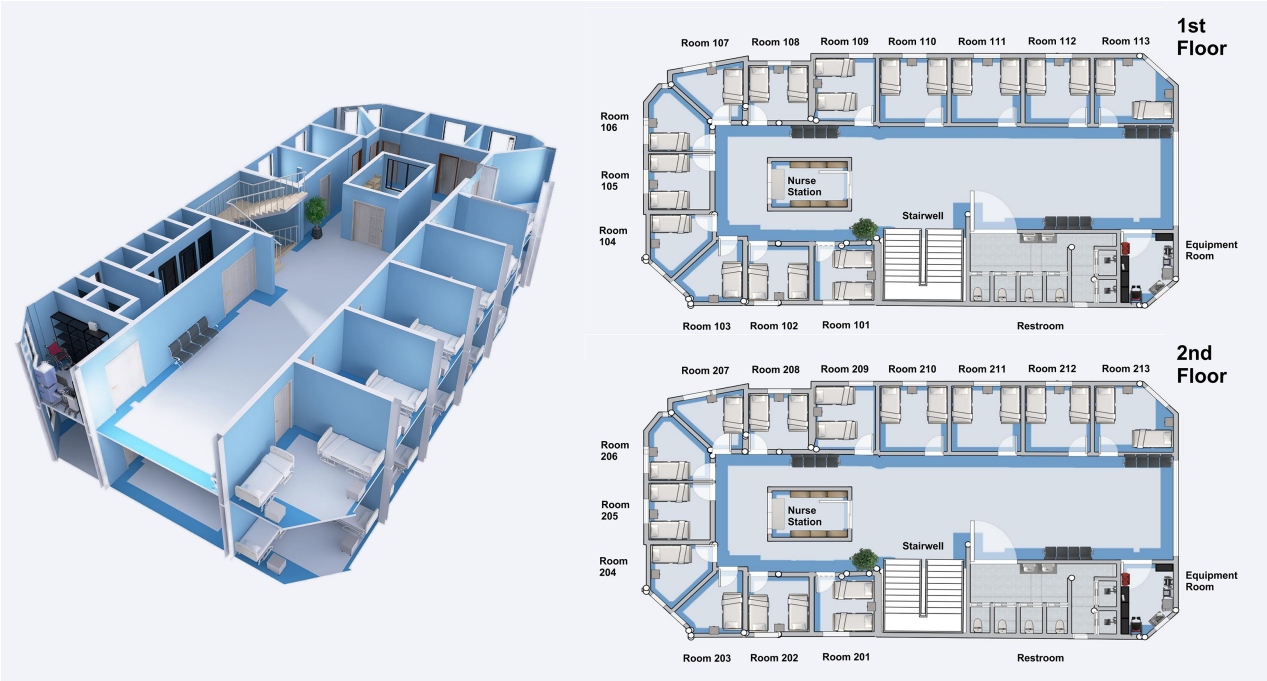


Figure 5.3: The hospital ward layout. Designed using an online tool on `www.homestyler.com`.

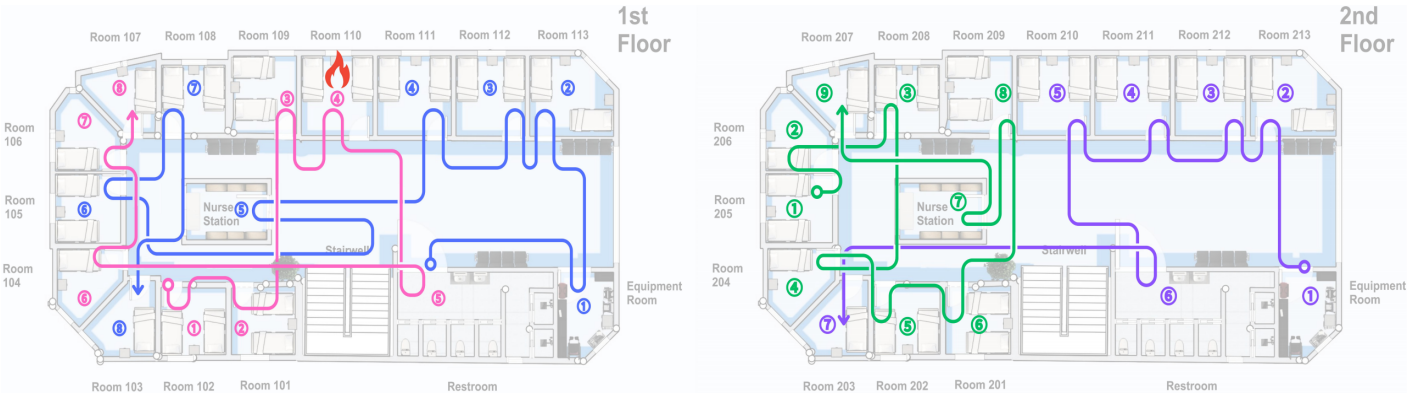
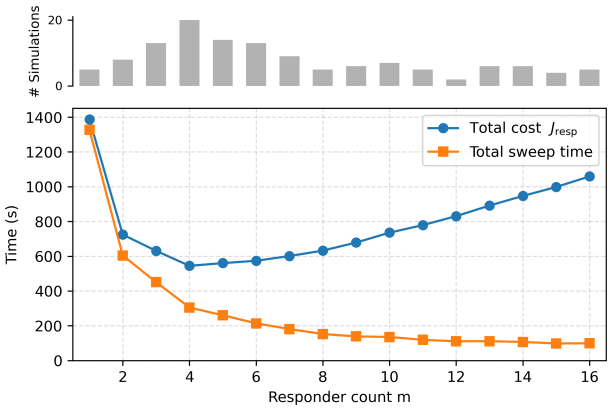


Figure 5.4: Sweeping paths and responder setup for Layout 2

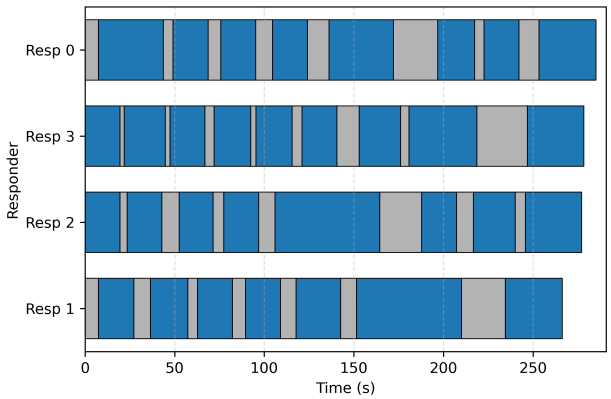
5.2 Layout 2: A Hospital Building

Fig. 5.3 illustrates a more complex multi-floor example: a two-level hospital ward with 32 rooms, including patient rooms, nurse stations, and equipment rooms. Because occupants are primarily weak patients or elderly individuals, the likelihood of non-ambulatory residents is substantially higher. We assume all patient-room doors remain unlocked for responder access, except for the nurse-station and equipment-room doors.

For the hospital ward, our outer optimizer recommends a team of **4 responders**. As shown in Fig. 5.5(a), an increase of m from 1 to 4 cuts the expected sweep time from about **22 min** (1300 s) to roughly **5 min**(300 s). After having 4 responders, additional responders yield only small further reductions while steadily increasing the total cost J_{resp} due to the per-responder penalty. The bar chart also shows that the bandit algorithm automatically focuses its simulation budget on responder counts near this optimal value. The timelines in Fig. 5.5(b) confirm our result that with four responders the sweep finishes in about 270 s. All responders stay busy and finish within a narrow time window, indicating that the planner balances workload effectively across the team.



(a) Number of simulations (top) and cost values (bottom) for possible responder counts



(b) Responder timelines for a specific G_1 setting. Grey indicates transit time and blue indicates service time.

Figure 5.5: Results for Layout 2.

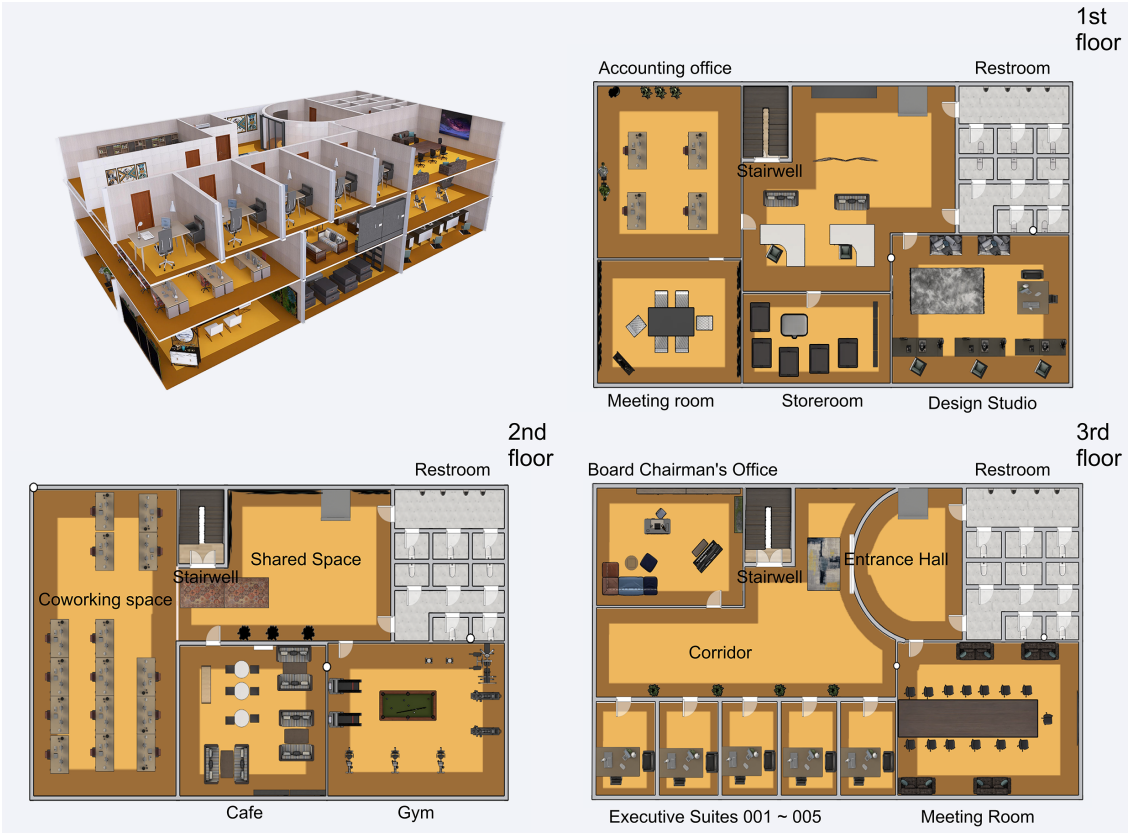


Figure 5.6: The multi-use office layout. Designed using an online tool on `www.homestyler.com`.

5.3 Layout 3: Multi-use Office Building

The multi-use complex is more challenging, spanning three floors and mixing offices, recreation rooms and gyms, making the distribution of occupants much more random and complicated. In this case the optimizer recommends **6 responders**, succeeding in an approximate **sixfold** efficiency gain: sweep time drops from about **42 min** (2500 s) with one responder to around **6.5 min** (400 s). As seen in Fig. 5.8(a), the bandit layer quickly focuses simulations on 5-7 responders and thus avoids oversizing and burdening the team. The resulting plan (Fig. 5.8(b)) has all six responders finishing in roughly 400 **s** with near to no idle time, showcasing the model's

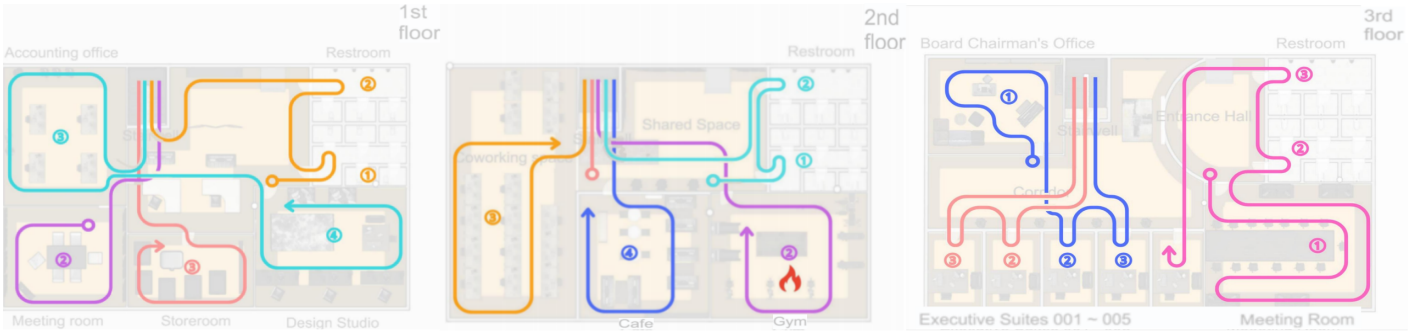
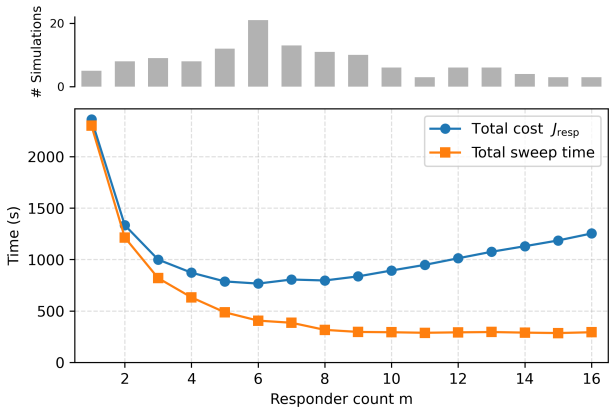
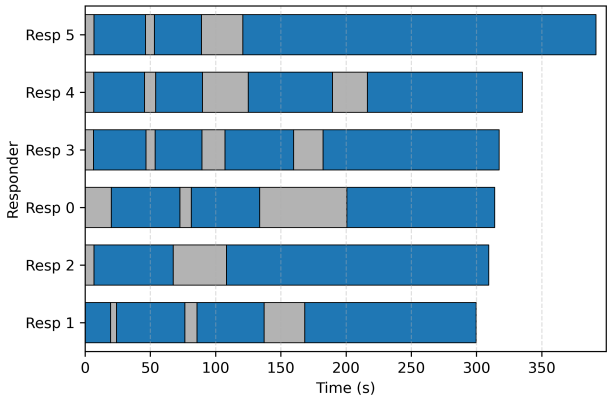


Figure 5.7: Sweeping paths and responder setup for Layout 3



(a) Number of simulations (top) and cost values (bottom) for possible responder counts



(b) Responder timelines for a specific G_1 setting. Grey indicates transit time and blue indicates service time.

Figure 5.8: Results for Layout 3

effectiveness when faced with the random distribution of occupants across the building.

5.4 Sensitivity Analysis

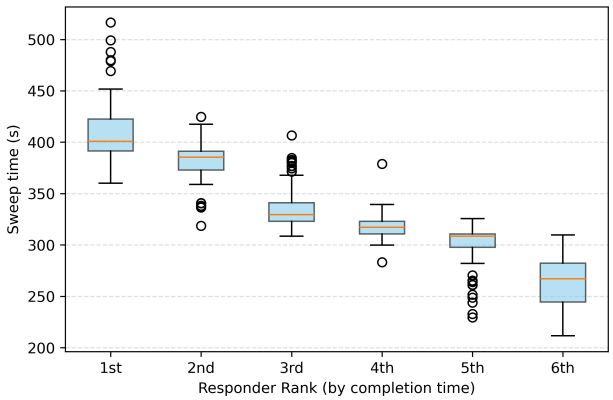
We evaluate the sensitivity of our sweep path and responder setting planner by repeatedly sampling building states G_ω from the prior G_2, treating each G_ω as a possible realization of the unknown ground truth G_1.

5.4.1 Sensitivity of Sweep Path Optimization

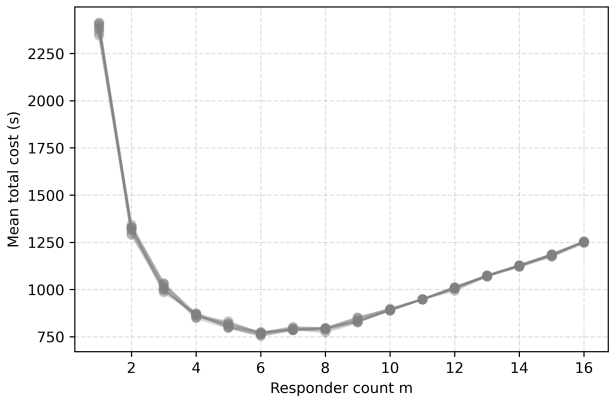
For a fixed responder configuration, we run the sweep-path planner under multiple sampled G_ω. Fig. 5.9(a) shows the distribution of completion time across these trials. Despite substantial variation in room conditions, the overall sweep-time overhead remains below 25%, demonstrating that the planner adapts effectively to unexpected scenarios.

5.4.2 Sensitivity of Responder Setting Optimization

Each optimization run estimates the expected total costs using a budget of 128 total simulations across all possible responder configurations. We repeat the full procedure 10 times, producing 10 curves of total cost versus responder count m. As illustrated in Fig. 5.9(b), these curves nearly coincide, indicating that 128 samples are sufficient to obtain stable estimates and consistent recommendations, while keeping computational cost manageable.

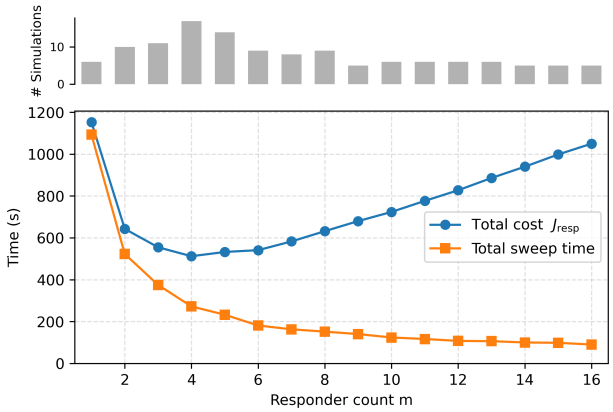


(a) Box plot for responder completion times with different G_1 (G_ω)

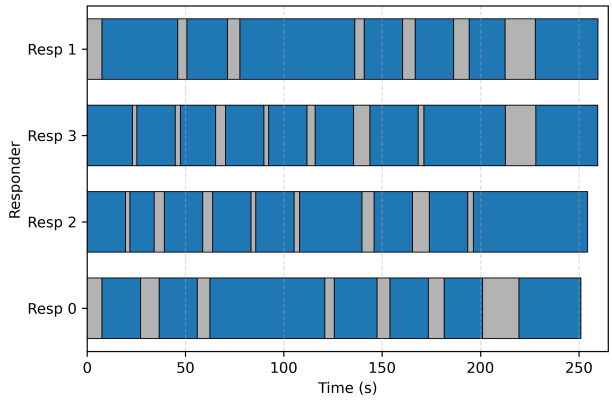


(b) Multiple runs of the entire optimization for responder setting.

Figure 5.9: Algorithm sensitivity analysis through multiple runs of sweep path planning and responder planning.



(a) Optimization for responder count



(b) Responder timelines for a specific G_1 setting

Figure 6.1: Parametric study for realistic factors: Results for gas leakage applied to hospital ward.

6 Realistic Factors and Additional Technologies

6.1 Parametric study: Odorless Gas Leakage

Our above-defined model has already accounted for many realistic constraints:

1. Smoke spreading and responders' decreased movement speed.
2. Prioritization of high-risk areas.
3. Occupants' awareness of an emergency.
4. Time varying hazards.
5. Non-standard building geometries (e.g., multi-use office layouts).

We conduct the following parametric study through another type of emergency that changes parts of these realistic factors to evaluate how these factors impact our model.

Odorless gas spreads freely through the building and is not detected by fire alarms. Compared to fire, occupants react more slowly but lose consciousness more quickly. The gas also imposes weaker movement slowdowns on responders. To reflect these differences on complicated factors we have considered through, we adjust our occupant-dynamics and hazard models as follows:

1. **Occupants' awareness of an emergency**: The transition rate from ambulatory to non-ambulatory $\lambda_{M\to N}(t)$ increases and the transition rates of evacuation $\lambda_{M\to O}(t)$ and $\lambda^{\text{peer}}_{N\to O}(t)$ decrease, which can be achieved by increasing c_1 and reducing c_2, c_3.
2. **Time-varying hazards**: Since the gas offers less physical obstruction, we decrease the scale constant σ (Eq. (3.19)) and set a lower value for β_{haz} (Eq. (3.20)).
3. **Hard-to-locate hazard source**: Because the leakage source is initially unknown, hazard proximity z^i_{haz} cannot be assigned at first. We therefore compute preliminary risk weights using only vulnerability z^i_{vul} and expected occupancy z^i_{occ} to plan initial routes and responder placement. Once the leakage source is identified, we update z^i_{haz} and rerun the full planning algorithm to reflect the true hazard distribution.

Comparing Fig. 6.1 with Fig. 5.5, we observe that replacing the fire with a gas leakage doesn't cause a significant change in total sweep time for near-optimal values of m, but increases its sample variance, causing the overall UCB simulation distribution in Fig. 6.1(a) to be smoother than that in Fig. 5.5(a).

6.2 Additional Technologies

Beyond our mathematical framework, several emerging technologies can further support sweeping operations. Broadly, these tools help in two ways: (1) enhancing the efficiency of responder movement within buildings; and (2) acquiring more knowledge of room conditions before inspection.

Improving movement efficiency

Technologies that enhance mobility can reduce transit time and improve responder safety:

- **Augmented Reality (AR) Glasses**: Provide real-time overlays of floor plans, optimal routes, exits, and hazard zones, giving responders immediate situational awareness while navigating complex layouts.
- **Smart Helmets**: Equipped with cameras and environmental sensors, these helmets offer hands-free navigation aids, localized alerts, and continuous tracking of responder position and surroundings.
- **Wearable Sensors**: Monitor responder physiology (e.g., fatigue, heart rate) and local environmental conditions, enabling early detection of dangerous situations and supporting safer movement.

Enhancing prior knowledge of rooms

Technologies that reveal room conditions ahead of time improve planning accuracy and reduce uncertainty:

- **Smart Environmental Sensors**: Installed throughout the building to detect smoke, gas concentration, heat, or motion, transmitting real-time alerts and helping responders prioritize critical rooms.
- **Building Information Modeling (BIM)**: Provides detailed digital building models—including layouts, materials, and known hazards—supporting better pre-entry planning and route selection.

To sum up, these technologies help accelerate evacuation and enhance operational safety by equipping responders with accurate, timely information, and ultimately saving lives and optimizing resources.

7 Advantages and Weaknesses

Advantages:

- **Real-time sweep planning.** When a responder becomes unexpectedly delayed (e.g., encountering an unusually large number of occupants), the planner dynamically reallocates remaining tasks to other responders, preserving overall efficiency and adaptability.
- **Balanced human-centered objectives.** The objective function simultaneously minimizes total sweep time and prioritizes high-risk rooms with vulnerable occupants, ensuring both operational efficiency and occupant safety.
- **Efficient responder-configuration search.** The upper-level optimizer uses an exploration–exploitation policy to evaluate many stochastic building states under a limited computational budget, yielding robust recommendations for responder count and positions.

- **Full factor integration.** Both planning layers incorporate hazards, human behavior, room properties, and dynamic updates by optimizing over end-to-end simulations rather than isolated components.

Weaknesses:

- **Building pre-processing.** The building layout must be digitized and stored in advance. Although many fire departments already use pre-fire planning tools [8], this requirement limits immediate applicability in poorly documented buildings.
- **Modeling assumptions.** The model focuses on occupant rescue and omits property protection. Several simplifying assumptions are made (Section 2), such as uncongested corridors and uniform hazard spread, which may not fully reflect extreme real-world conditions.
- **Limited probabilistic space.** The prior space in G_2 for initial occupants $(M(0), N(0))$ is limited and may miss extreme out-of-distribution scenarios, though it is easily extensible to account for such cases.

8 Conclusion

We presented a comprehensive modeling and planning framework that (1) encodes building layout, occupant information, and prior probabilistic knowledge of sweep time using graph-based representations incorporating key real-world factors (Section 3); (2) enables dynamic, real-time sweep path planning through greedy initialization and local-search refinement, balancing overall sweep efficiency with protection of vulnerable occupants (Section 4.1); and (3) optimizes responder deployment using a risk-aware greedy start room strategy combined with a computational-budget–aware multiarmed bandit method (Section 4.2).

Given a fire alarm as the emergency, the application of our model on a provided single-layer office layout, a two-floor hospital ward, and a three-floor multi-use office building proves that our model effectively and efficiently optimizes the sweep path and responder settings that result in robustly improving sweep time while ensuring priority of high-risk rooms with non-ambulatory occupants and hazards (Section 5). We have introduced sufficient realistic factors, allowing us to successfully adapt our model to odorless gas leakage, where realistic factors change accordingly (Section 6).

There exist more dangerous and special emergencies, such as **indiscriminate shooting** in schools, considering a *moving* shooter that requires much more complicated information and optimization strategies. Regrettably, we did not delve into them due to time limit, but it is of high worth to adapt our model to such situations.

References

[1] https://www.usfa.fema.gov/statistics/deaths-injuries/
[2] https://en.wikipedia.org/wiki/Graph_theory
[3] D. T. Gillespie, "A general method for numerically simulating the stochastic time evolution of coupled chemical reactions," *Journal of Computational Physics*, vol. 22, no. 4, pp. 403–434, 1976.
[4] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, no. 2–3, pp. 235–256, 2002.
[5] https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm
[6] https://en.wikipedia.org/wiki/Multi-armed_bandit
[7] J.-Y. Audibert, S. Bubeck, and R. Munos, "Exploration–exploitation trade-off using variance estimates in multi-armed bandits," in *Proceedings of the 23rd Conference on Neural Information Processing Systems (NIPS)*, pp. 49–56, 2009.
[8] https://www.auburnal.gov/fire/pre-fire-planning/
[9] https://machinelearningmastery.com/statistical-data-distributions/
[10] https://en.wikipedia.org/wiki/Gamma_distribution
[11] G. Mosheiov, "The travelling salesman problem with pick-up and delivery," *European Journal of Operational Research*, vol. 79, no. 2, pp. 299–310, 1994.

Appendix

Parameters and sample outputs for three layouts.

Layout 1

```
=== Realization G_omega: room parameters and initial occupants ===
R_L1: area=20.0m², per=18.0m, clutter=0.15, vul=0, E[occ]=1.3, risk_w=2.11, lock_prob=0.30, M(0)=0, N(0)=0
R_L2: area=20.0m², per=18.0m, clutter=0.15, vul=0, E[occ]=1.3, risk_w=2.11, lock_prob=0.30, M(0)=0, N(0)=0
R_L3: area=20.0m², per=18.0m, clutter=0.15, vul=0, E[occ]=1.3, risk_w=2.11, lock_prob=0.30, M(0)=0, N(0)=0
R_R1: area=20.0m², per=18.0m, clutter=0.15, vul=0, E[occ]=1.3, risk_w=2.11, lock_prob=0.30, M(0)=0, N(0)=0
R_R2: area=20.0m², per=18.0m, clutter=0.15, vul=0, E[occ]=1.3, risk_w=2.11, lock_prob=0.30, M(0)=0, N(0)=0
R_R3: area=20.0m², per=18.0m, clutter=0.15, vul=0, E[occ]=1.3, risk_w=2.11, lock_prob=0.30, M(0)=0, N(0)=0

=== Simulation result for optimal configuration ===
Estimated sweep makespan: 77.1 s
Objective value: 137.0

Responder 0:
  [   0.0,    0.0]  Travel R_L1 -> R_L1 via ['R_L1']
  [   0.0,   19.5]  Sweep room R_L1
  [  19.5,   27.4]  Travel R_L1 -> R_L2 via ['R_L1', 'T0', 'T1', 'R_L2']
  [  27.4,   45.2]  Sweep room R_L2
  [  45.2,   49.3]  Travel R_L2 -> R_R1 via ['R_L2', 'T1', 'R_R1']
  [  49.3,   69.6]  Sweep room R_R1

Responder 1:
  [   0.0,    0.0]  Travel R_R3 -> R_R3 via ['R_R3']
  [   0.0,   19.5]  Sweep room R_R3
  [  19.5,   27.8]  Travel R_R3 -> R_R2 via ['R_R3', 'T3', 'T2', 'R_R2']
  [  27.8,   53.2]  Sweep room R_R2
  [  53.2,   57.6]  Travel R_R2 -> R_L3 via ['R_R2', 'T2', 'R_L3']
  [  57.6,   77.1]  Sweep room R_L3
```

Layout 2

```
=== Realization G_omega: room parameters and initial occupants ===
F1_R1: area=8.1m², per=11.4m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=2.25, lock_prob=0.20, M(0)=1, N(0)=0
F1_R2: area=9.3m², per=11.5m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=2.25, lock_prob=0.20, M(0)=1, N(0)=0
F1_R3: area=7.7m², per=11.1m, clutter=0.30, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F1_R4: area=7.4m², per=10.9m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F1_R5: area=8.6m², per=11.7m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F1_R6: area=7.8m², per=11.2m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F1_R7: area=7.3m², per=10.8m, clutter=0.30, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F1_R8: area=8.2m², per=11.5m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F1_R9: area=9.3m², per=12.2m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=2.25, lock_prob=0.20, M(0)=1, N(0)=0
F1_R10: area=10.8m², per=13.2m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=2.25, lock_prob=0.20, M(0)=1, N(0)=0
F1_R11: area=11.2m², per=13.4m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=2.25, lock_prob=0.20, M(0)=1, N(0)=0
F1_R12: area=10.5m², per=13.0m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=2.25, lock_prob=0.20, M(0)=1, N(0)=0
F1_R13: area=11.7m², per=13.7m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=2.25, lock_prob=0.20, M(0)=1, N(0)=0
F1_R14: area=8.3m², per=11.5m, clutter=0.70, vul=2, E[occ]=1.0, risk_w=5.25, lock_prob=0.50, M(0)=1, N(0)=0
F1_R15: area=21.6m², per=18.6m, clutter=0.40, vul=1, E[occ]=2.0, risk_w=4.00, lock_prob=0.20, M(0)=2, N(0)=0
F1_R16: area=8.3m², per=11.5m, clutter=0.00, vul=0, E[occ]=2.0, risk_w=2.50, lock_prob=0.60, M(0)=2, N(0)=0
F2_R1: area=8.1m², per=11.4m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F2_R2: area=8.3m², per=11.5m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F2_R3: area=7.7m², per=11.1m, clutter=0.30, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F2_R4: area=7.4m², per=10.9m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F2_R5: area=8.6m², per=11.7m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F2_R6: area=7.8m², per=11.2m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F2_R7: area=7.3m², per=10.8m, clutter=0.30, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F2_R8: area=8.2m², per=11.5m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F2_R9: area=9.3m², per=12.2m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F2_R10: area=10.8m², per=13.2m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F2_R11: area=11.2m², per=13.4m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F2_R12: area=10.5m², per=13.0m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F2_R13: area=11.7m², per=13.7m, clutter=0.20, vul=0, E[occ]=1.0, risk_w=1.25, lock_prob=0.20, M(0)=1, N(0)=0
F2_R14: area=8.3m², per=11.5m, clutter=0.70, vul=2, E[occ]=1.0, risk_w=4.25, lock_prob=0.50, M(0)=0, N(0)=1
F2_R15: area=21.6m², per=18.6m, clutter=0.40, vul=1, E[occ]=2.0, risk_w=3.00, lock_prob=0.20, M(0)=2, N(0)=0
F2_R16: area=8.3m², per=11.5m, clutter=0.00, vul=0, E[occ]=2.0, risk_w=1.50, lock_prob=0.60, M(0)=2, N(0)=0

=== Simulation result for optimal configuration ===
Responder count m= = 4
Start positions: ['F1_R15', 'F2_R15', 'F1_R2', 'F2_R5']
Sweep makespan: 285.1 s
Objective (makespan + risk term): 1023.7 s
Responder completion time variance: 60.54

Responder 0:
  [   0.0,    7.6]  Travel F1_R15 -> F1_R14 via ['F1_R15', 'F1_T2', 'F1_T1', 'F1_R14']
  [   7.6,   43.7]  Sweep room F1_R14
  [  43.7,   49.2]  Travel F1_R14 -> F1_R13 via ['F1_R14', 'F1_T1', 'F1_R13']
  [  49.2,   68.7]  Sweep room F1_R13
  [  68.7,   75.8]  Travel F1_R13 -> F1_R12 via ['F1_R13', 'F1_T2', 'F1_R12']
  [  75.8,   95.3]  Sweep room F1_R12
  [  95.3,  104.7]  Travel F1_R12 -> F1_R11 via ['F1_R12', 'F1_T3', 'F1_R11']
  [ 104.7,  124.2]  Sweep room F1_R11
  [ 124.2,  136.1]  Travel F1_R11 -> F1_R16 via ['F1_R11', 'F1_T4', 'F1_R16']
  [ 136.1,  172.1]  Sweep room F1_R16
  [ 172.1,  196.8]  Travel F1_R16 -> F1_R5 via ['F1_R16', 'F1_T4', 'F1_T8', 'F1_T7', 'F1_R5']
  [ 196.8,  217.5]  Sweep room F1_R5
  [ 217.5,  222.7]  Travel F1_R5 -> F1_R8 via ['F1_R5', 'F1_T6', 'F1_R8']
  [ 222.7,  242.2]  Sweep room F1_R8
  [ 242.2,  253.3]  Travel F1_R8 -> F1_R3 via ['F1_R8', 'F1_T6', 'F1_T7', 'F1_R3']
  [ 253.3,  285.1]  Sweep room F1_R3
  Rooms swept (order): F1_R14 -> F1_R13 -> F1_R12 -> F1_R11 -> F1_R16 -> F1_R5 -> F1_R8 -> F1_R3

Responder 1:
  [   0.0,    7.6]  Travel F2_R15 -> F2_R14 via ['F2_R15', 'F2_T2', 'F2_T1', 'F2_R14']
  [   7.6,   27.2]  Sweep room F2_R14
  [  27.2,   36.5]  Travel F2_R14 -> F2_R13 via ['F2_R14', 'F2_T1', 'F2_R13']
```

(The rest is omitted due to space limit)

Layout 3

```
=== Realization G_omega: room parameters and initial occupants ===
f1R1: area=85.0m², per=36.0m, clutter=0.40, vul=0, E[occ]=1.0, risk_w=1.00, lock_prob=0.20, M(0)=1, N(0)=0
f1R2: area=61.0m², per=36.0m, clutter=0.30, vul=0, E[occ]=1.0, risk_w=1.00, lock_prob=0.20, M(0)=1, N(0)=0
f1R3: area=45.0m², per=30.0m, clutter=0.70, vul=2, E[occ]=4.0, risk_w=1.00, lock_prob=0.50, M(0)=3, N(0)=1
f1R4: area=90.0m², per=40.0m, clutter=0.60, vul=1, E[occ]=4.0, risk_w=1.00, lock_prob=0.20, M(0)=4, N(0)=0
f1R5: area=27.0m², per=20.0m, clutter=0.00, vul=0, E[occ]=2.0, risk_w=1.00, lock_prob=0.60, M(0)=2, N(0)=0
f1R6: area=27.0m², per=20.0m, clutter=0.00, vul=0, E[occ]=2.0, risk_w=1.00, lock_prob=0.60, M(0)=2, N(0)=0
f2R1: area=135.0m², per=60.0m, clutter=0.30, vul=1, E[occ]=12.0, risk_w=1.50, lock_prob=0.10, M(0)=12, N(0)=0
f2R2: area=74.0m², per=40.0m, clutter=0.30, vul=0, E[occ]=5.0, risk_w=1.00, lock_prob=0.10, M(0)=5, N(0)=0
f2R3: area=90.0m², per=50.0m, clutter=0.50, vul=1, E[occ]=8.0, risk_w=1.50, lock_prob=0.40, M(0)=8, N(0)=0
f2R4: area=27.0m², per=20.0m, clutter=0.00, vul=0, E[occ]=2.0, risk_w=1.00, lock_prob=0.60, M(0)=2, N(0)=0
f2R5: area=27.0m², per=20.0m, clutter=0.00, vul=0, E[occ]=2.0, risk_w=1.00, lock_prob=0.60, M(0)=2, N(0)=0
f3R1: area=56.0m², per=30.0m, clutter=0.10, vul=0, E[occ]=1.0, risk_w=1.00, lock_prob=0.20, M(0)=1, N(0)=0
f3R2: area=17.5m², per=15.0m, clutter=0.10, vul=0, E[occ]=3.0, risk_w=1.00, lock_prob=0.20, M(0)=3, N(0)=0
f3R3: area=17.5m², per=15.0m, clutter=0.10, vul=0, E[occ]=3.0, risk_w=1.00, lock_prob=0.20, M(0)=3, N(0)=0
f3R4: area=17.5m², per=15.0m, clutter=0.10, vul=0, E[occ]=3.0, risk_w=1.00, lock_prob=0.20, M(0)=3, N(0)=0
f3R5: area=17.5m², per=15.0m, clutter=0.10, vul=0, E[occ]=3.0, risk_w=1.00, lock_prob=0.20, M(0)=3, N(0)=0
f3R6: area=17.5m², per=15.0m, clutter=0.10, vul=0, E[occ]=3.0, risk_w=1.00, lock_prob=0.20, M(0)=3, N(0)=0
f3R7: area=90.0m², per=40.0m, clutter=0.10, vul=0, E[occ]=8.0, risk_w=1.00, lock_prob=0.20, M(0)=8, N(0)=0
f3R8: area=27.0m², per=20.0m, clutter=0.00, vul=0, E[occ]=2.0, risk_w=1.00, lock_prob=0.60, M(0)=2, N(0)=0
f3R9: area=27.0m², per=20.0m, clutter=0.00, vul=0, E[occ]=2.0, risk_w=1.00, lock_prob=0.60, M(0)=2, N(0)=0

=== Simulation result for optimal configuration ===
Responder count m= = 6
Start positions: ['f2R1', 'f3R1', 'f1R1', 'f3R7', 'f2R3', 'f1R4']
Sweep makespan: 377.1 s
Objective (makespan + risk term): 732.2 s
Responder completion time variance: 838.20

Responder 0:
  [   0.0,   20.2]  Travel f2R1 -> f3R3 via ['f2R1', 'f2E', 'f3E', 'f3T4', 'f3T3', 'f3R3']
  [  20.2,   72.7]  Sweep room f3R3
  [  72.7,   81.4]  Travel f3R3 -> f3R2 via ['f3R3', 'f3T3', 'f3T2', 'f3R2']
  [  81.4,  132.8]  Sweep room f3R2
  [ 132.8,  199.2]  Travel f3R2 -> f1R3 via ['f3R2', 'f3T2', 'f3T3', 'f3T4', 'f3E', 'f2E', 'f1E', 'f1R1', 'f1R3']
  [ 199.2,  295.0]  Sweep room f1R3
  Rooms swept (order): f3R3 -> f3R2 -> f1R3

Responder 1:
  [   0.0,    0.0]  Travel f3R1 -> f3R1 via ['f3R1']
  [   0.0,   19.5]  Sweep room f3R1
  [  19.5,   24.1]  Travel f3R1 -> f3R4 via ['f3R1', 'f3T4', 'f3R4']
  [  24.1,   76.6]  Sweep room f3R4
  [  76.6,   97.1]  Travel f3R4 -> f3R8 via ['f3R4', 'f3T4', 'f3E', 'f3T7', 'f3T8', 'f3R8']
  [  97.1,  136.8]  Sweep room f3R8
  [ 136.8,  176.0]  Travel f3R8 -> f2R2 via ['f3R8', 'f3T8', 'f3T7', 'f3E', 'f2E', 'f2R2']
  [ 176.0,  307.4]  Sweep room f2R2
  Rooms swept (order): f3R1 -> f3R4 -> f3R8 -> f2R2

Responder 2:
  [   0.0,    6.9]  Travel f1R1 -> f1R2 via ['f1R1', 'f1R2']
  [   6.9,   69.6]  Sweep room f1R2
  [  69.6,  110.9]  Travel f1R2 -> f2R3 via ['f1R2', 'f1R1', 'f1E', 'f2E', 'f2T1', 'f2R3']
  [ 110.9,  311.8]  Sweep room f2R3
  Rooms swept (order): f1R2 -> f2R3

Responder 3:
  [   0.0,    6.6]  Travel f3R7 -> f3R9 via ['f3R7', 'f3T8', 'f3R9']
  [   6.6,   42.6]  Sweep room f3R9
  [  42.6,   56.5]  Travel f3R9 -> f3R6 via ['f3R9', 'f3T8', 'f3T7', 'f3T6', 'f3R6']
  [  56.5,  109.0]  Sweep room f3R6
  [ 109.0,  119.6]  Travel f3R6 -> f3R5 via ['f3R6', 'f3T6', 'f3T5', 'f3R5']
  [ 119.6,  172.1]  Sweep room f3R5
  [ 172.1,  195.6]  Travel f3R5 -> f3R7 via ['f3R5', 'f3T5', 'f3T7', 'f3T8', 'f3R7']
```

(The rest is omitted due to space limit)

Report on Use of AI Tools

We utilized GitHub Copilot to assist programming via automatic completion and debugging. This is done in Microsoft Visual Studio Code with the following extensions:

- *GitHub Copilot* (Oct 25 version): `https://marketplace.visualstudio.com/items?itemName=GitHub.copilot`
- *GitHub Copilot Chat* (Oct 25 version and Nov 18 version): `https://marketplace.visualstudio.com/items?itemName=GitHub.copilot-chat`

Models including GPT-4.1, Grok Code Fast 1, Claude 4.5 Sonnet, and GPT-5-Codex in the chat extension are used for assistance on code debugging.